

Projet de Système de Gestion de Base de Données Relationnelle en OCaml

Mathéo LECOMPAGNON

Mars 2026

Table des matières

1	Introduction	1
2	Fonctionnalités Principales	2
2.1	Gestion des Tables	2
2.2	Fonctions de base	2
2.2.1	Insertion (insert)	2
2.2.2	Produit Cartésien (prod)	2
2.2.3	Projection (projection)	3
2.2.4	Restriction (restrict)	3
2.3	Normalisation	3
2.3.1	Fonction compute_deps	3
2.3.2	Fonction compute_elementary_deps	4
2.3.3	Fonction normalization_level	4
3	Limites du Programme	6
4	Conclusion	6
5	Documentation	6

1 Introduction

Ce projet implémente un système de gestion de bases de données relationnelles simplifié en OCaml, avec support des opérations fondamentales comme la sélection, la projection, le produit cartésien, et l'analyse des dépendances fonctionnelles.

2 Fonctionnalités Principales

2.1 Gestion des Tables

- **Structure des données :**
 - Type `dbtype` pour les types de données (`TInt`, `TText`)
 - Type `dbvalue` pour les valeurs (`VInt` of `int`, `VText` of `string`, `VNull`)
 - Structure de table avec `schema` et `row`
 - **Validation des tables (`check_table`) :** Pour déterminer la validité d'une table, il a fallu définir une table correcte :
 - Chaque ligne doit être de même taille que la liste des champs
 - Les types de données de chaque case doit correspondre au type de donnée dans la liste des champs : `VInt` pour `TInt`, `VText` pour `TText`
 - Une valeur nulle peut être présente dans une case seulement si le champ i correspondant l'autorise, c'est à dire le booléen du champ $b_i = \text{true}$
 - Il y a unicité des noms de champ : chaque champ admet son nom
- Notre algorithme se résume à vérifier la taille de chaque ligne une par une. Puis de vérifier pour chaque champ, si la valeur associée dans chaque ligne vérifie les contraintes du champ. Enfin, de vérifier la présence de plusieurs champs avec le même nom dans la table.

2.2 Fonctions de base

2.2.1 Insertion (`insert`)

- Ajout d'une ligne valide à une table, au début de la table.
- Vérification des contraintes de validité sur les lignes, et sur la table. En effet, nous avons développé la validité d'une table et des lignes dans 2.1.

2.2.2 Produit Cartésien (`prod`)

Notre algorithme est le suivant :

On vérifie d'abord la validité des deux tables en entrée.

pour chaque ligne de la deuxième table, on effectue le produit cartésien de cette ligne avec la première table. On rassemble ces lignes pour former la liste des lignes de la table produit.

Enfin, on calcule la concaténation entre les champs de la première table et de la deuxième. Tout en prenant en compte les conflits de noms ne garantissant pas l'unicité des noms de champ. A chaque fois qu'on ajoute un champ à la concaténation et que son nom est déjà pris, on ajoute un "2" et vérifie récursivement si son nom est déjà pris. Par exemple, la concaténation de `nom`, `nom2` avec `nom` donne `nom`, `nom2`, `nom22`. Cette concaténation est la liste des champs de la table produit.

Les limites de ce programme est dans la concaténation des listes de champs, qui utilise l'opérateur `@` en augmentant la complexité temporelle de l'algorithme.

2.2.3 Projection (projection)

Notre algorithme consiste à :

Vérifier la validité de la table `tbl` et de la liste de champs `fields` en entrée, c'est à dire si le champ appartient bien à la table.

Pour chaque ligne l_i , on projette les champs `fields` sur la ligne l_i et la liste de champs de la table. On fait attention à vérifier si la liste de retour de ces projections est vide ou non, car cela nous permet de retourner une table vide lorsque la projection est vide au lieu d'avoir une liste de listes vide.

La projection des champs `fields` sur la ligne l_i consiste à faire la projection de chaque champ de `fields` sur la ligne l_i puis de concaténer dans le bon ordre.

Pour chaque champ de `fields`, on parcourt l_i et le schéma de la table en parallèle. Cela nous permet d'isoler la case où les noms de champs coïncident.

La complexité de cet algorithme montre ses limites, mais suit la logique de programmation fonctionnelle comparé à une approche avec des indices.

2.2.4 Restriction (restrict)

Il suffit de vérifier la validité de la table d'entrée, puis de parcourir chaque ligne en vérifiant la fonction de `test` en entrée

Nous avons implémenté le filtrage des lignes selon des critères Quelques comparaisons :

- Champ `=`, `<`, `>`, `<=`, `>=` valeur implémentés par `test...fv`
- Champ `=`, `<`, `>`, `<=`, `>=` champ implémentés par `test...ff`

En remplaçant `"..."` par `eq`, `lt`, `gt`, `le`, `ge` respectivement aux opérations présentées ci-dessus.

On considérera que la fonction d'entrée `test` est de type `schema -> row -> bool` car cela permet de filtrer les valeurs de la ligne mais également les noms de la table.

Nous aurions pu également implémenter des fonctions de test comme `IN`, `BETWEEN`, ou les opérateurs logiques comme `AND`, `OR` et `NOT` pour combiner intelligemment les fonctions de test élémentaires.

2.3 Normalisation

2.3.1 Fonction `compute_deps`

Pour calculer toutes les dépendances fonctionnelles de la table, nous faisons toutes les combinaisons possibles de champs. Puis on vérifie parmi ces combinaisons, lesquelles sont des dépendances fonctionnelles.

Pour rappel, une dépendance fonctionnelle est (lhs, rhs) telle que, pour chaque ligne $l_i, l_j, i \neq j$, on a :

$$lhs_i = lhs_j \implies rhs_i = rhs_j$$

avec lhs_x : liste de valeurs dans la table des champs de la liste lhs et rhs_x : liste de valeurs dans la table des champs de la liste rhs

Notre algorithme est le suivant :

On vérifie la validité de la table.

Puis on génère tous les sous ensembles possibles des noms de champs.

Pour chaque paire possible de sous-ensembles (lhs, rhs), on vérifie si c'est une dépendance fonctionnelle. C'est à dire pour une potentielle dépendance fonctionnelle (lhs, rhs), on parcourt chacune des lignes une à une, et les compare à tout autre ligne ayant la même valeur de lhs en vérifiant qu'elles admettent la même valeur de rhs.

La complexité de cet algorithme est très élevée :

- Génération de 2^m sous-ensembles pour m champs (pour lhs et rhs)
- Vérification de $O(n^2)$ pour chaque dépendance candidate (où n est le nombre de lignes)

Ce qui montre les limites de notre approche pour les tables avec beaucoup de champs ou de lignes.

2.3.2 Fonction `compute_elementary_deps`

Pour calculer toutes les dépendances fonctionnelles élémentaires d'une table, on doit identifier parmi toutes les dépendances fonctionnelles, lesquelles sont élémentaire.

Une dépendance est considérée comme élémentaire si :

- rhs contient exactement un champ
- rhs n'appartient pas à lhs
- Aucun sous-ensemble strict de lhs ne forme une dépendance valide avec le même rhs.

On doit alors calculer chaque sous-ensemble de lhs. Ce qui pose des problèmes de complexité exponentielle en fonction de la taille de lhs. En plus de la complexité du calcul initial de toutes les dépendances ($O(4^m)$).

2.3.3 Fonction `normalization_level`

Notre algorithme est le suivant :

On vérifie que la table est valide

On calcule toutes les clés candidates qui vont nous servir à savoir si la table est 3NF ou 2NF.

On vérifie alors si la table est 3NF en utilisant `check_nf3`. Elle parcourt chaque dépendance fonctionnelle élémentaire du type $X \rightarrow a_i$ et vérifie que soit X est une clé, soit a_i appartient à une des clés candidates calculées préalablement.

Si c'est pas le cas, on vérifie si elle est 2NF en utilisant `check_nf2`. On parcourt chaque champ a_i de la table, on vérifie que a_i n'appartient pas à une clé,

Puis nous calculons toutes les clés candidates de la table avec `list_all_keys`.

Ensuite, nous vérifions d'abord si la table est en 3NF en utilisant `check_nf3`. Si ce n'est pas le cas, nous vérifions si elle est en 2NF avec `check_nf2`. Par défaut, nous considérons qu'elle est au moins en 1NF.

La complexité de cet algorithme dépend fortement de l'efficacité des fonctions `check_nf3` et `check_nf2`. La détection des dépendances transitives pour la 3NF est particulièrement complexe et nécessite une liste complète et correcte des clés candidates.

Les limites de cette approche apparaissent lorsque nous devons distinguer clairement entre 1NF et 2NF dans certains cas limites, ou lorsque la table contient des dépendances fonctionnelles complexes.

La difficulté de cette fonction est de trouver des exemples de 1NF qui ne sont pas 2NF et 2NF mais pas 3NF et 3NF. Une table vide est considérée comme 3NF.

nom	profession	salaire
A	prof	12
B	prof	12
A	agent	13
A	nettoyage	13

TABLE 1 – table 1NF mais pas 2NF

Cette table est 1NF car chaque champ est indivisible, ce qui est supposé vrai pour toutes les tables dans notre projet. Mais elle n'est pas 2NF car :

$\text{nom, profession} \rightarrow \text{salaire}$

$\text{profession} \rightarrow \text{salaire}$

id_ordonnance	n_secu	nom
1	50	A
2	30	B
3	50	A

TABLE 2 – table 2NF mais pas 3NF

Cette table est 2NF. Mais n'est pas 3NF car :

- $\text{id_ordonnance} \rightarrow \text{nom}$
- id_ordonnance n'est pas une clé
- nom n'appartient pas à une clé

nom	niveau	arme
Lucas	10	épée
Claus	15	épée
Ness	20	batte

TABLE 3 – table 3NF

3 Limites du Programme

Les limites du programmes se trouvent dans les algorithmes naïfs utilisés, de grande complexité pour gérer les tables et vérifier les dépendances fonctionnelles et la normalisation. Les tests effectués ne sont pas exhaustifs.

4 Conclusion

Ce projet a permis d’implémenter les fondamentaux des bases de données en Ocaml. Malgré ses limitations, il permet de manipuler les bases de données et travailler avec la normalisation. Plusieurs pistes d’améliorations peuvent être mises en avant comme l’optimisation des algorithmes naïfs pour gérer de grandes bases de données. Nous aurions pu également implémenter plus de fonctions de test pour la fonction **restrict** comme IN, BETWEEN, ou les opérateurs logiques comme OR, AND, NOT. Les jointures ou les agrégations sont aussi une piste de développement qui permettrait de compléter ce projet. On pourrait également implémenter une interface utilisateur pour faciliter la création des bases de données et leur manipulation.

5 Documentation

Fonction `check_table`

Signature : <code>table -> bool</code>
Description : Vérifie qu’une table est bien formée.
Préconditions : rien
Paramètres : — <code>tbl</code> : table à tester
Retourne : retourne la condition “[tbl] est une table valide”
Exceptions : aucune exception

Fonction insert

Signature : table -> row -> table
Description : Insère une ligne dans une table.
Préconditions : rien
Paramètres : — <code>tbl</code> : table où l'on doit insérer — <code>row</code> : ligne à insérer
Retourne : insère si possible <code>[row]</code> dans la table <code>[tbl]</code>
Exceptions : — <code>Invalid_table</code> si <code>[tbl]</code> n'est pas une table bien formée — <code>Invalid_row_size(actuel, attendu)</code> si <code>[row]</code> n'a pas la bonne taille — <code>Invalid_row_type</code> si <code>[row]</code> n'admet pas le bon typage

Fonction prod

Signature : table -> table -> table
Description : Calcule le produit cartésien de deux tables.
Préconditions : rien
Paramètres : — <code>tbl1</code> : première table du produit — <code>tbl2</code> : deuxième table du produit
Retourne : renvoie la table produit cartésien entre les tables <code>[tbl1]</code> et <code>[tbl2]</code>
Exceptions : — <code>Invalid_table</code> si au moins une des tables <code>[tbl1]</code> et <code>[tbl2]</code> n'est pas une table bien formée

Fonction projection

Signature : table -> string list -> table
Description : Effectue une projection sur une table.
Préconditions : rien
Paramètres : — <code>tbl</code> : table de données — <code>fields</code> : liste des noms de champs à projeter
Retourne : retourne la projection des champs <code>[fields]</code> sur la table <code>[tbl]</code>
Exceptions : — <code>Invalid_table</code> si la table <code>[tbl]</code> n'est pas bien formée — <code>Invalid_field</code> si un champ de <code>[fields]</code> n'appartient pas à la table <code>[tbl]</code>

Fonction restrict

Signature : <code>table -> (schema -> row -> bool) -> table</code>
Description : Filtre les lignes d'une table selon un critère de sélection. Ne conserve que les lignes satisfaisant la condition spécifiée.
Préconditions : La table doit être valide (vérifiée par <code>check_table</code>)
Paramètres : — <code>tbl</code> : table à filtrer — <code>test</code> : fonction de test définissant le critère de sélection
Retourne : Nouvelle table contenant uniquement les lignes qui satisfont le critère
Exceptions : — <code>Invalid_table</code> si la table n'est pas valide
Utilisation : Pour utiliser cette fonction, combinez-la avec les fonctions de test fournies : — Comparaison champ-valeur : — <code>test_eq_fv "champ" valeur</code> : <code>champ = valeur</code> — <code>test_lt_fv "champ" valeur</code> : <code>champ < valeur</code> — <code>test_gt_fv "champ" valeur</code> : <code>champ > valeur</code> — <code>test_le_fv "champ" valeur</code> : <code>champ ≤ valeur</code> — <code>test_ge_fv "champ" valeur</code> : <code>champ ≥ valeur</code> — Comparaison champ-champ : — <code>test_eq_ff "champ1" "champ2"</code> : <code>champ1 = champ2</code> — <code>test_lt_ff "champ1" "champ2"</code> : <code>champ1 < champ2</code> — <code>test_gt_ff "champ1" "champ2"</code> : <code>champ1 > champ2</code> — <code>test_le_ff "champ1" "champ2"</code> : <code>champ1 ≤ champ2</code> — <code>test_ge_ff "champ1" "champ2"</code> : <code>champ1 ≥ champ2</code> Exemples d'utilisation : — Filtrer les lignes où le champ "hp" est supérieur à 90 : <code>restrict tbl (test_gt_fv "hp" (VInt 90))</code> — Filtrer les lignes où le champ "nom" est égal à "Ness" : <code>restrict tbl (test_eq_fv "nom" (VText "Ness"))</code> — Filtrer les lignes où le champ "hp" est égal au champ "pp" : <code>restrict tbl (test_eq_ff "hp" "pp")</code> — Combiner plusieurs conditions : <code>restrict tbl (fun cols row -></code> <code> (test_gt_fv "hp" (VInt 90) cols row) &&</code> <code> (test_eq_fv "arme" (VText "batte") cols row))</code>

Fonction `compute_deps`

Signature : <code>table -> (string list * string list) list</code>
Description : Calcule toutes les dépendances fonctionnelles d'une table.
Préconditions : rien
Paramètres : — <code>tbl</code> : table à tester
Retourne : retourne la liste de toutes les dépendances fonctionnelles trouvées de la table <code>[tbl]</code>
Exceptions : — <code>Invalid_table</code> si la table <code>[tbl]</code> n'est pas bien formée

Fonction `compute_elementary_deps`

Signature : <code>table -> (string list * string list) list</code>
Description : Calcule les dépendances fonctionnelles élémentaires d'une table.
Préconditions : rien
Paramètres : — <code>tbl</code> : table de données
Retourne : retourne la liste de toutes les dépendances fonctionnelles élémentaires de la table <code>[tbl]</code>
Exceptions : — <code>Invalid_table</code> si la table <code>[tbl]</code> n'est pas bien formée

Fonction `normalization_level`

Signature : <code>table -> int</code>
Description : Détermine le niveau de normalisation d'une table.
Préconditions : rien
Paramètres : — <code>tbl</code> : table à vérifier
Retourne : retourne le niveau de normalisation de <code>[tbl]</code> sous forme d'un entier entre 1 et 3
Exceptions : — <code>Invalid_table</code> si <code>[tbl]</code> n'est pas une table bien formée